

数据表示

Menghong Yu, EECS, PKU

2026/10/1

© SHANGHAI HENIAN INFORMATION TECHNOLOGY CO.,LTD

基本概念

basic concepts

位（bit）：数据存储的最小单位

字节（Byte）：最小的可寻址的存储器单位，1 Byte = 8 bits

十六进制

- `0x` 开头
- `0~9，A~F`，不区分大小写
- 一个字节的值域是 00_{16} ~ FF_{16}

```
0xA = 10 = 0b1010
0xC = 12 = 0b1100
0xF = 15 = 0b1111
```

*需要熟练掌握进制转换

基本概念

basic concepts

字长（word size）：处理器一次自然处理的数据宽度，也是指针和地址表示能力的重要参数

- 若虚拟地址实际使用 w 位，则理论地址范围为 $[0, 2^w - 1]$，最多可区分 2^w 个字节地址
- 机器“标称 64 位”不代表实现了一整个 2^{64} 字节虚拟地址空间；实际地址宽度还受处理器实现和操作系统限制

字节顺序

- 小端序（little endian）：最低有效字节在最前面
- 大端序（big endian）：最高有效字节在最前面

一定要先想清楚字节是怎么排序的（哪边是小地址哪边是高地址），再去辨析大端/小端序

字节是一个整体，不会说字节内部是按照小端序还是大端序排列

数组总是小下标对应低地址，大下标对应高地址

字节顺序

byte order

同样是表示 `0x12345678`，在不同的机器上，内存中的存储可能是：

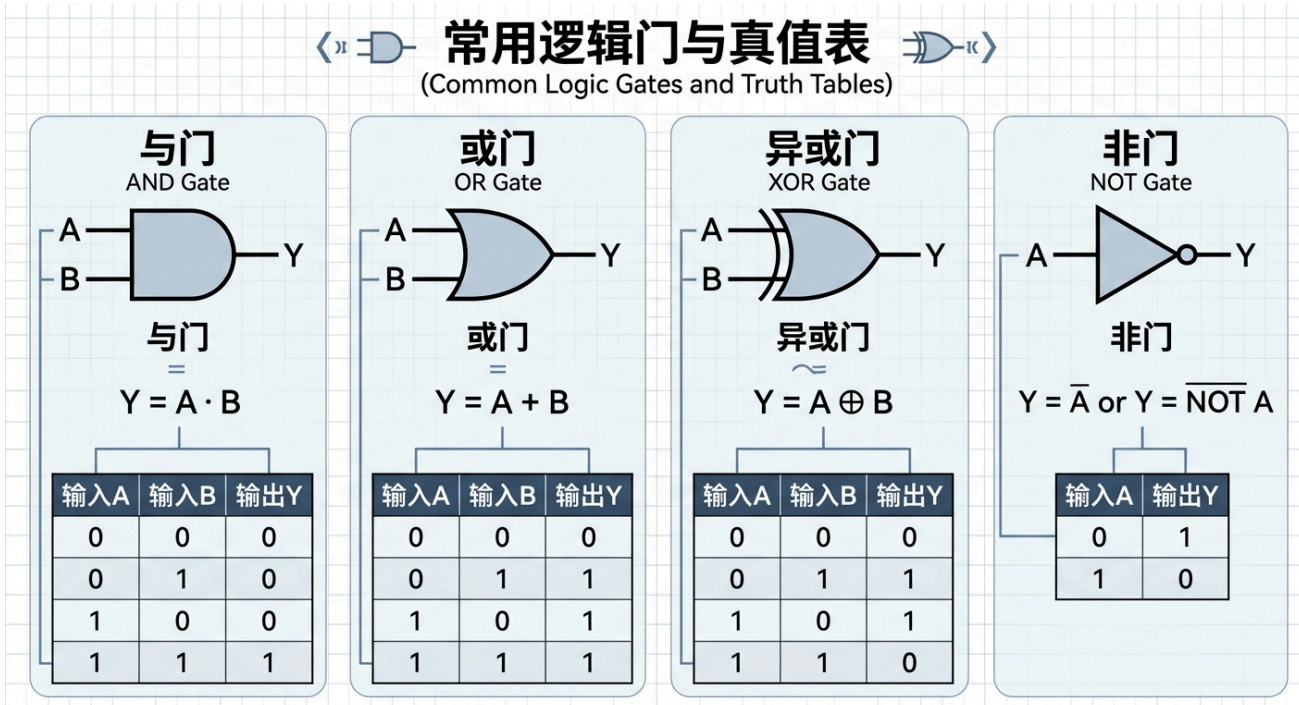
- 小端序：（低地址）`0x78 0x56 0x34 0x12`（高地址）
- 大端序：（低地址）`0x12 0x34 0x56 0x78`（高地址）

应用

- 现今常见的 x86-64 和 RISC-V 系统通常采用 **小端序**。
  - 将一个对象的地址转换成指向更小整数类型的指针时，小端序下该地址仍指向原对象的最低有效字节
- 网络协议通常使用 **大端序**，也称为网络字节序。

布尔运算

boolean operation



逻辑运算

logical operation

- 逻辑运算符：`&&`(and，与)、`||`(or，或)、`!`(not，非)
- 逻辑运算符的运算结果只有两种：`true` 和 `false`

注意与**按位**运算区分！

短路特性

- `&&`：前一个表达式为假时，后一个表达式不执行，因为无论后一个表达式是什么，结果都是假
- `||`：前一个表达式为真时，后一个表达式不执行，因为无论后一个表达式是什么，结果都是真

也即：如果对第一个参数求值能确定表达式的结果，那么逻辑运算符不会对第二个参数求值

Q：为什么 `p && *p++` 不会引用空指针？

整数数据类型

integer type

- 接下来我们将讨论以下几种整数数据类型:
  - bool: 往往使用1个byte表示, 使用 `0x00` 表示false, 使用 `0x01` 表示true.
    - 为什么明明一个bit就可以表示bool, 却还要占用一个byte呢?
    - 因为计算机内存的最小可寻址单位是byte
  - char: 使用1个byte表示.
    - 常用转换规则: `A: 0x41` , `a: 0x61` , `0: 0x30` .
  - short(i16): 使用2个byte表示, 数据范围$[-2^{15}, 2^{15}-1]$.
    - unsigned short(u16): 使用2个byte表示, 数据范围$[0, 2^{16}-1]$.
  - int(signed)(i32): 使用4个bytes表示, 数据范围$[-2^{31}, 2^{31}-1]$.
    - unsigned int(unsigned)(u32): 使用4个bytes表示, 数据范围$[0, 2^{32}-1]$.
  - long long(i64): 使用8个bytes表示, 数据范围$[-2^{63}, 2^{63}-1]$.
    - unsigned long long(u64): 使用8个bytes表示, 数据范围$[0, 2^{64}-1]$.

无符号整数数据类型

unsigned integer type

> 在下文中，为了严格区分"底层字节表示"与"实际数学数值"，我们引入符号 $[\![A]\!] = a$，表示二进制数据 A 所映射/解读出的真实数值为 a.
>
> 如果你学过离散/数理逻辑/程序语言等相关内容，你可能会管 `A` 叫做语法(Syntax)侧，而 `a` 叫做语义(Semantics)侧.

- 整数有无穷多个，可计算机内存总是有限个.
  - 内存中不可能表示任意大的整数!
- 假设我们现在有w个bits可供我们表示数字:
  - 如何表示自然数(无符号整数)?
    - 使用常规的二进制表示，考虑$x = [x_{w-1}, x_{w-2}, ..., x_0]$，它表示的无符号数字正是:
    - $[\![x]\!]_U = \sum_{i=0}^{w-1}(x_i \times 2^i)$
  - 如何表示自然数(无符号整数)的加法?
    - 两个 w 位的无符号数相加，结果的范围是 $[0, 2^{w+1} - 2]$，而这需要 $w + 1$ 位来表示
    - 如果两个数a, b加起来后，最高位是0，那我们的表示仍然适用!
      - 例如: `0x10+0x01->0x11`
    - 如果两个数a, b加起来后，最高位是1呢? ~~那我们的表示就完蛋子~~

无符号整数数据类型

unsigned integer type

- 假设我们现在有w个bits可供我们表示数字:
  - 如何表示自然数(无符号整数)的加法?
    - 如果两个数a, b加起来后, 二进制下有$w+1$位呢?
    - 我们可以直接丢掉最高位(溢出)! 这样剩下的就只有w位了.
      - 例如(假设$w=8$): `0x10+0xf1->0x01`
    - 这实际上就是对2^w取模操作!

无符号整数数据类型与模运算

unsigned integer type and mode

假设无符号类型的两个变量A, B, 满足$[\![A]\!] = a, [\![B]\!] = b$.

记$add(A, B)$为二者相加的结果. 假设$a, b < 2^w$.

无符号加法

- 如果$a + b < 2^w$, 则$[\![add(A, B)]\!] = a + b$
- 如果$a + b \geq 2^w$, 则$[\![add(A, B)]\!] = a + b - 2^w$

模意义下加法

- 如果$a + b < 2^w$, 则$(a + b) \bmod 2^w = a + b$
- 如果$a + b \geq 2^w$, 则$(a + b) \bmod 2^w = a + b - 2^w$

有符号整数数据类型

signed integer type

- 假设我们现在有w个bits可供我们表示数字:
  - 如何表示负数?
  - 如果一个数$+n=0$,那这个数就可以用来表示$-n$.
  - 考虑加法的溢出:
    - `0xff+0x01->0x00`
    - 我们可以定义`0xff`为-1.
  - 具体而言, 如果$[\![x]\!]_I=n$且$n\neq T_{min}$, 我们记$[\![(\sim x)+1]\!]_I=-n$.
    - 对于最小补码值 T_{min}，其相反数无法在相同位宽的有符号类型中表示；位级取反加一仍得到原位模式
    - 也可以记作: $[\![x]\!]_I=-x_{w-1}\times 2^{w-1}+\sum_{i=0}^{w-2}(x_i\times 2^i)$
  - 这就是所谓的"补码".

有符号数与无符号数之间的转换

conversion between signed and unsigned

在相同位宽的对应有符号/无符号类型之间转换时，位模式保持不变，**只改变解释这些位的方式**

- 回忆一个二进制表示 $x = [x_{w-1}, x_{w-2}, ..., x_0]$:
  - 它代表的无符号数: $[\![x]\!]_U = \sum_{i=0}^{w-1}(x_i \times 2^i)$
  - 它代表的有符号数: $[\![x]\!]_I = -x_{w-1} \times 2^{w-1} + \sum_{i=0}^{w-2}(x_i \times 2^i)$
- 容易见到:
  - 当最高位为0时, 它们代表的数字相同;
  - 当最高位为1时, 它们代表的数字差了2^w.
- 事实上总满足: $[\![x]\!]_U \equiv [\![x]\!]_I \pmod{2^w}$

有符号数与无符号数之间的转换

conversion between signed and unsigned

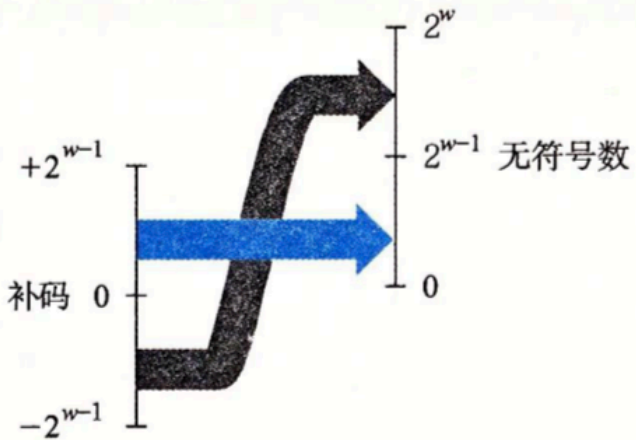


图 2-17 从补码到无符号数的转换。函数 *T2U* 将负数转换为大的正数

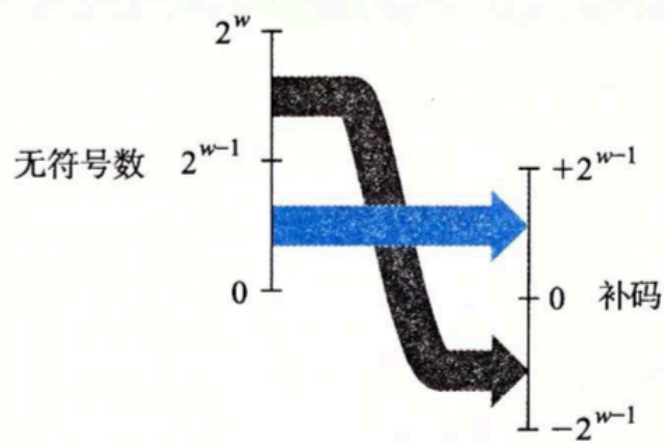


图 2-18 从无符号数到补码的转换。函数 *U2T* 把大于 $2^{w-1}-1$ 的数字转换为负值

位移

bit shift

- 左移：`<<`，右移：`>>`

逻辑右移 / 算术右移

- 逻辑右移：左边补 0
- 算术右移：左边补 最高位
- 逻辑左移 / 算术左移：右边补 0

C 语言：有符号数算术右移（但注意没有明确规定，只是大家都这么做），无符号数逻辑右移

> 这使得C语言中，无论a是有符号还是无符号，`a>>k` 总等价于 $\lfloor \frac{a}{2^k} \rfloor$
>
> 注意：C语言中的除法是向0取整，所以 `(-3)/2 == -1` 而 `(-3)>>1 == -2`

运算时，需要注意运算符优先级 （如果记不住就全加上括号！）

例如：`1 << 2 + 3 << 4`， （记忆方法：回忆大一写计概的时候的 `cout << a + b`，因为C++ 中流相关运算符就重载了位移）

关于移 k 位而 k 很大？参见课本 p41 旁注，注意这个在 C 语言中是 UB。

扩展和截断

extension and truncation

扩展一个数

- 对于无符号数，高位补 0
- 对于有符号数（补码），高位**补符号位**

当把 short（有符号数，2 字节）强制转换为 unsigned（无符号数，4 字节）时?

先进行数位扩展，再转换为无符号数

为什么对于有符号数，高位补符号位?

设现在位数为 m，补到 n 位，且 $n > m$，则:

$$-2^{n-1} + \sum_{i=m-1}^{n-2} 2^i = -2^{m-1}$$

扩展和截断

extension and truncation

截断一个数

- 对于无符号数 $x = [x_{w-1}, ..., x_0]$，截断为 w' 位，则 $x' = [x_{w'-1}, ..., x_0]$

  即 $x' = x \mod 2^{w'}$

- 补码截断，原理上与无符号数类似，但对于数位的解释方式不同（最高位符号位）

扩展和截断

extension and truncation

在采用小端法存储机器上运行下面的代码，输出的结果将会是?

（如无特别说明，`int`，`unsigned` 为 32 位长，`short` 为 16 位长，0~9 的 ASCII 码分别是 `0x30 ~ 0x39`，之后题目同）

```c
#include <stdio.h>

int main() {
    char* s = "2018";
    int* p1 = (int*)s;
    short s1 = (*p1) >> 12;
    unsigned u1 = (unsigned)s1;
    printf("0x%x \n", u1);
}
```

1. `char *` 声明的是一个指针，底层是 C 风格的字符串，以 `\0` 结尾，所以实际存储中，其指向一个 `char` 数组的首字节，该数组在内存中是连续存储的，从低地址到高地址依次为 `0x32`，`0x30`，`0x31`，`0x38`，`0x00`
2. `p1` 强制将 `s` 看做一个 `int` 型指针，由于采用小端法，所以其指向的内存被看做了一个 `int` 型变量，其值为 `0x38313032`
3. 将 `int` 型变量 `0x38313032` 右移 12 位，得到 `0x00038313`，然后截断，得到 `short` 型变量 `s1` 的值 `0x8313`
4. 从短、有符号的 `short` 型变量 `s1` 到长、无符号的 `unsigned` 型变量 `u1`，先符号扩展，再看作无符号数，得到 `0xffff8313`

乘除法

multiplication and division

无符号数乘法

等于乘法计算得到的结果（一个 $2w$ 位长的数），而后截断到 w 位

补码乘法

等于有符号数乘法后得到的结果（一个 $2w$ 位长的数），再按位截断到 w 位

核心：粗暴的位级表示截断

乘以二的幂次

乘以 2^k 时，**相当于左移 k 位**

整数类型的加/减/乘

add/sub/mul

在固定位宽的机器整数运算中，加/减/乘可以视作 $\mathrm{mod} 2^w$ 意义下的运算，因此有：

- 加法交换律: $[\![A + B]\!] = [\![B + A]\!]$
- 加法结合律: $[\![(A + B) + C]\!] = [\![A + (B + C)]\!]$
- 乘法交换律: $[\![A * B]\!] = [\![B * A]\!]$
- 乘法结合律: $[\![(A * B) * C]\!] = [\![A * (B * C)]\!]$
- 乘法分配律: $[\![(A + B) * C]\!] = [\![(A * C) + (B * C)]\!]$

整数类型的除/模

div/mod

当除数不为 0，且不出现有符号的 $T_{min}/-1$ 溢出时，C 整数除法向零取整：

- 除法的结果是向零取整, 具体地:
  - 如果 `A,B` 同号, 则$[\![A/B]\!] = \lfloor \frac{[\![A]\!]}{[\![B]\!]} \rfloor$
  - 如果 `A,B` 异号, 则$[\![A/B]\!] = \lceil \frac{[\![A]\!]}{[\![B]\!]} \rceil$
- 商和余数满足 $A = (A/B) \times B + A\%B$，因此 $A\%B = A - (A/B) \times B$
- `7/3 == 2, 7%3 == 1`
- `(-7)/3 == -2, (-7)%3 == -1`
- `7/(-3) == -2, 7%(-3) == 1`
- `(-7)/(-3) == 2, (-7)%(-3) == -1`

除数为 0 是未定义行为；对于补码有符号整数，$T_{min}/-1$ 的数学结果不可表示，在 C 中同样是未定义行为。

判断溢出

determine overflow

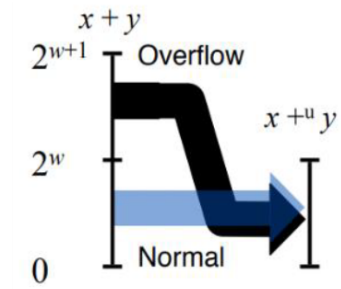
无符号数加法

- $s = x + y$，若 $s < x$ 且 $s < y$ 则溢出

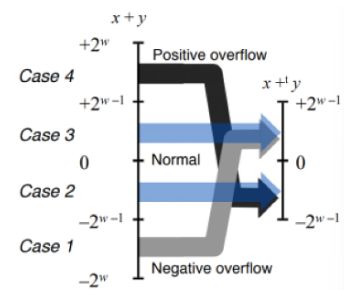
有符号数加法

- $s = x + y$，若 $x > 0$ 且 $y > 0$ 且 $s \leq 0$，则正溢出；
- 若 $x < 0$ 且 $y < 0$ 且 $s \geq 0$，则负溢出

为什么只有这两种情况?



无符号数加法



有符号数加法

数据的其他表示

other representations

反码

最高位的权重为 $-2^{w-1}+1$

在此方法下，0 的表示有两种：$000...0$ 和 $111...1$

原码

最高位是符号位，用来确定剩下的位应该取正权还是负权

$$\mathrm{B2S}_w(x) = (-1)^{x_{w-1}} \times \sum_{i=0}^{w-2}(x_i \times 2^i)$$

经典案例：浮点数

其它语言中的整数类型

- Rust: 摒弃C语言中混乱的类型名, 统一使用形如`i32,u16`形式的整数类型名
- Python：`int` 原生支持任意精度。以常见 64 位 CPython 构建为例，`PyLongObject` 使用可变长“数字”数组，通常以 2^{30} 为基数；这是实现细节，并非 Python 语言规范
- Haskell：区分固定精度的 `Int` 和任意精度的 `Integer`。以 GHC 为例，`Integer` 的大整数后端可使用 GMP 或原生实现，并非语言层面固定为 GMP

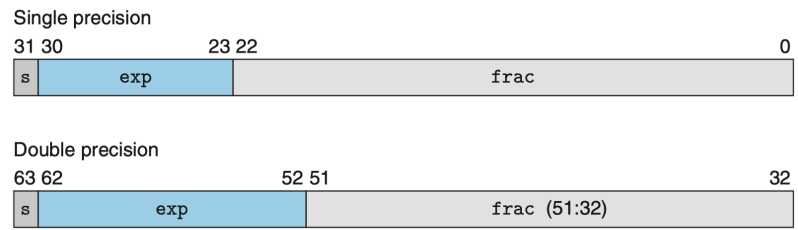
浮点数表示

floating point

IEEE 754 标准

本质是对实数的近似

- 符号位：1 位
- 指数位：$w_e(e)$ 位
- 尾数位：m 位



实际表示：

- 单精度浮点数（`float`）：$w_e = 8,\ m = 23$
- 双精度浮点数（`double`）：$w_e = 11,\ m = 52$

各个位长多少要牢牢记清楚！考试会考！

注意，0 的表示有两种，+0 和 -0

理解 IEEE 754

underlying IEEE 754

规格化值($e_{raw} \in [1, 2^{w_e} - 2]$)

$$V = (-1)^s \times (1.M) \times 2^E$$

- s 为符号位，M 为尾数，E 为实际指数
- $E = e_{raw} - Bias$，其中 e_{raw} 为阶码处这 w_e 位的实际值，$Bias = 2^{w_e - 1} - 1$
- 尾数隐含了 1

非规格化值($e_{raw} = 0$)

$$V = (-1)^s \times (0.M) \times 2^E$$

- $E = 1 - Bias$, 虽然此时 $e_{raw} = 0$, 但如此规定可以实现规格化值和非规格化值的平滑过渡
- 尾数没有隐含的 1
- 0也是非规格化值, 有$+0, -0$两种表达方式

理解 IEEE 754

underlying IEEE 754

无穷($e_{raw} = 2^{w_e} - 1, M = 0$)

- 符号位 s 区分正无穷/负无穷
- 表示无穷大，常由非零有限数除以零或浮点溢出产生；$0/0$、$\infty - \infty$ 等无效运算产生 `NaN`

非数值($e_{raw} = 2^{w_e} - 1, M \neq 0$)

- 符号位 s 任意
- NaN 用于表示未定义或无法表示的值，例如 $0/0$ 或 $\sqrt{-1}$
- 通常在比较中被认为不等于任何值，包括自身

特殊值的应用

- 处理异常情况：如除零错误、溢出、下溢等
- 提高计算的鲁棒性，避免程序崩溃
- 在数值算法中用于标记和处理特殊情况

理解 IEEE 754

underlying IEEE 754

平滑过渡

考虑一种特殊的浮点数，$e = 2$，$m = 3$

```
0 01 000

0 00 111
```

$$Bias = 2^{e-1} - 1 = 2^{2-1} - 1 = 1$$

$$1.000_2 \times 2^{01_2 - 1} = 1.000_2$$

$$0.111_2 \times 2^{1-1} = 0.111_2$$

浮点舍入

rounding

由于浮点数是对实数的近似，浮点数可以精确表示的实数是有限的，所以舍入是不可避免的。

实数舍入到浮点数

- 舍入到最近值，恰好居中时取偶数（round-to-nearest, ties-to-even）
- 类比“四舍六入五成双”，避免统计偏差

以下十进制例子演示“舍入到最近整数”：

$$
\begin{aligned}
1.234 &\Rightarrow 1.0 \\
1.678 &\Rightarrow 2.0 \\
1.500 &\Rightarrow 2.0
\end{aligned}
$$

> 对1.11舍入最低位会得到什么结果? 舍入最低两位呢? 对1.101舍入最低位会得到什么结果? 舍入最低三位呢?

浮点数转整型

- C 的浮点数转整数会丢弃小数部分，即向零截断：`-9.9 -> -9`，`9.9 -> 9`

浮点舍入

rounding

采用 round-to-nearest, ties-to-even，且舍入结果为有限 `float` 时，实数转换为单精度浮点数的绝对误差上界是多少?

1. 显然阶码位要尽可能的大，才会让误差变大，由于不考虑 `Inf`，所以最大时，阶码位为 `11111110`，即 2^8-2，于是 $E = e_{raw} - Bias = (2^8-2) - (2^7-1) = 2^7 - 1 = 127$
2. 尾数位的最大误差发生在舍入时，尾数的 23 位的最后 1 位是上取了还是下取了，最大误差为 $2^{-23} \cdot \frac{1}{2} = 2^{-24}$
3. 于是绝对误差上界为 $2^{127} \times 2^{-24} = 2^{103}$

也即，原数的精确表示实际上是:

$$\pm 2^{127} \times (1.\underbrace{111...1}_{22个1}01)_2$$

其被舍入到了

$$\pm 2^{127} \times (1.\underbrace{111...1}_{22个1}0)_2$$

浮点运算

floating point arithmetic

- 加法可交换 `x + y = y + x`，加法不可结合 `(x + y) + z != x + (y + z)`，`NaN` 没有加法逆元
- 浮点加法具有非严格单调性：若 $a \geq b$，且两次加法均不产生 `NaN`，则 $x + a \geq x + b$
- 乘法可交换 `x * y = y * x`，乘法不可结合 `(x * y) * z != x * (y * z)`
- 乘法在加法上不可分配 `(x + y) * z != x * z + y * z`
- 小心特殊值：`+inf`，`-inf`，`NaN`
  - 设计NaN的 `!=` 表达式(例如 `NaN != NaN` 以及 `NaN != 1`)为真，其他涉及 `NaN` 的比较表达式都为假。
  - `-inf == -inf < R < +inf == +inf`
  - `1/(+0) == +inf`，`1/(-0) == -inf`
  - `1/(+inf) == +0`，`1/(-inf) == -0`

IEEE 754 异常

exception

- Invalid operation：零乘无穷、零除零、无穷除无穷、无穷减无穷等，默认结果为 `NaN`
- Division by zero：非零有限数除以零，默认结果为带符号的无穷
- Overflow：精确结果幅值过大；默认舍入模式下通常得到带符号的无穷，其他舍入模式也可能得到最大有限值
- Underflow：舍入后的结果极小且不精确，可能得到非规格化数或零
- Inexact：舍入结果与精确结果不同

更多详情请见 IEEE754 异常处理

补充: 模型训练中的浮点数

除了我们刚刚讨论的float类型(或称FP32)和double类型(或称FP64), 近年还涌现出一大批针对大模型训练设计的浮点数类型:

- IEEE 754标准本身就规定了FP16浮点类型, 它由1个符号位，5位指数位和10位小数位组成
- 近年也开始流行 FP8 格式，常见变体包括 E4M3（常用于前向传播的权重和激活）和 E5M2（常用于反向传播的梯度）。

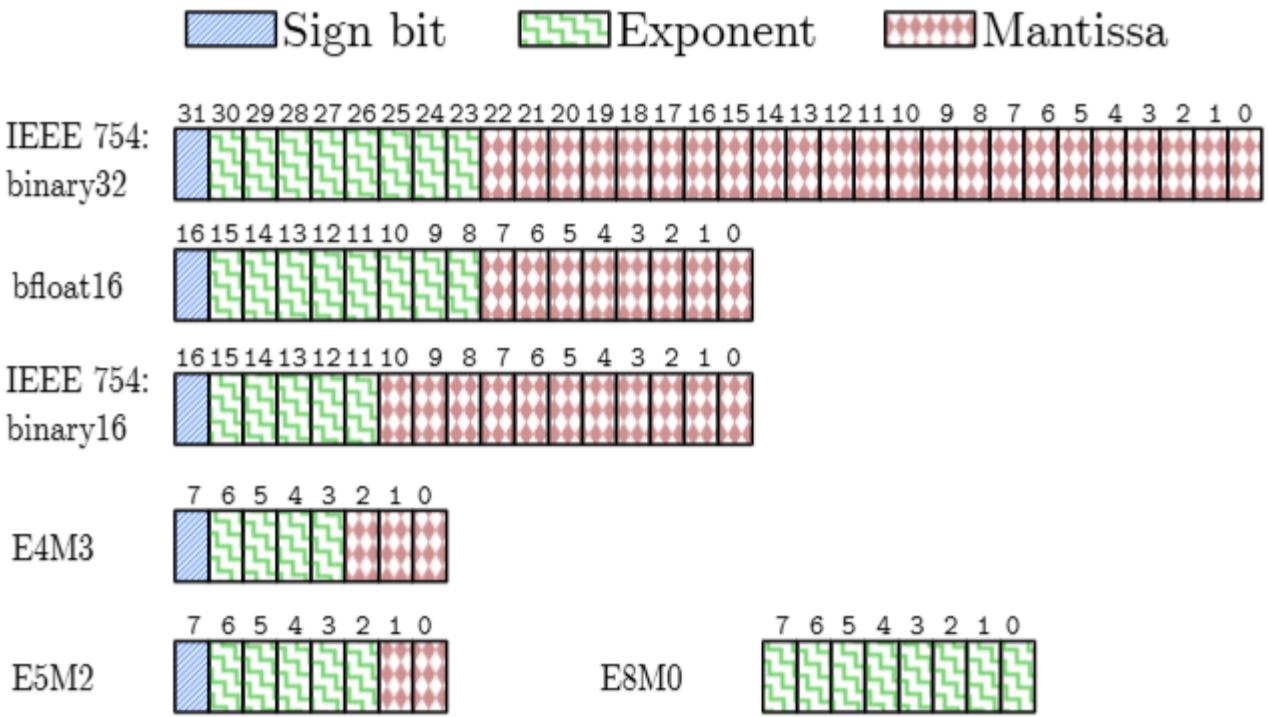
> 两个浮点数相乘后，精确乘积通常需要更多有效位，最终仍要按目标格式舍入；降低乘法输入精度可以减少硬件开销

- 英伟达提出了 TF32 运算格式。数据仍以 FP32 存储；参与 Tensor Core 乘法时，输入按 TF32 精度舍入到 10 位小数字段，乘积以 FP32 精度累加

> 一些情景对尾数精度要求不高, 但要求可以表示很大范围的数字

- Google Brain提出了BF16浮点类型, 它由1个符号位，8位指数位（和FP32一致）和7位小数位（低于FP16）组成

补充: 模型训练中的浮点数



补充: 模型训练中的浮点数

- 《AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration》
- 《Microscaling Data Formats for Deep Learning》
- 《BitNet: Scaling 1-bit Transformers for Large Language Models》
- 《The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits》

练习：字节顺序

Exercises

1. 在 x86-64 机器上运行如下代码，输出是？
2. 如果把 B 的初始值改为 `0x34566666`，输出是？

```c
#include <stdio.h>

int main() {
  unsigned int A = 0x11112222;
  unsigned int B = 0x33336666;
  void *x = (void *)&A;
  void *y = 2 + (void *)&B;
  unsigned short P = *(unsigned short *)x;
  unsigned short Q = *(unsigned short *)y;
  printf("0x%04x", P + Q);
  return 0;
}
```

考虑 A 和 B 在内存中的存储，从低地址到高地址，A 为 `22 22 11 11`，B 为 `66 66 33 33`。P 显然为 `22 22`，Q 为 `33 33`，所以结果为 `0x5555`。

改初值之后，B 为 `66 66 56 34`，因此 `P = 0x2222`，`Q = 0x3456`，二者相加得到 `0x5678`。

练习：字节顺序

Exercises

在 x86-64 机器上运行如下代码，输出是？（提示：`0` 的 ASCII 码为 `0x30`）

```
#include <stdio.h>

int main() {
  char A[12] = "11224455";
  char B[12] = "11445577";
  void *x = (void *)&A;
  void *y = 2 + (void *)&B;
  unsigned short P = *(unsigned short *)x;
  unsigned short Q = *(unsigned short *)y;
  printf("0x%04x", Q - P);
  return 0;
}
```

注意字符串在内存中的存储形式。P 为 `31 31`，Q 为 `34 34`，Q - P 得到 `03 03`，所以输出 `0x0303`。

整型/类型提升

Integral/Type Promotion

下面这个程序的输出是?

```c
#include <stdio.h>

int main() {
  unsigned char uc = 128;
  signed char c = -128;
  printf("%d %d\n", uc == c, uc + c);
}
```

整型/类型提升

Integral/Type Promotion

- 规则 #1：对 `char`、`signed char`、`unsigned char`、`short` 等类型进行整数提升时，若 `int` 能表示原类型的全部值，则提升为 `int`；否则提升为 `unsigned int`

> `unsigned char` 是怎么扩展到 `int` 的?

- 规则 #2：int 及以上，存在一系列按等级排列的类型。当表达式中数“是否有符号”相同，但等级不同时，统一扩展到存在的最高等级类型
  - 类型等级为：`int` < `long` < `long long`
  - 因为 C 标准中规定了 `sizeof(int) <= sizeof(long) <= sizeof(long long)`

整型/类型提升

Integral/Type Promotion

- 规则 #3：无符号类型与相应有符号类型处于同一等级。表达式中无符号数的等级更高，或二者相同时，有符号类型统一转换到无符号
  - 例如：
    - `int` 和 `unsigned`：统一转换为 `unsigned`
    - `int` 和 `unsigned long`：统一转换为 `unsigned long`
    - `-1 < sizeof(int)` 为假：为什么？参考：https://en.cppreference.com/w/c/types/size_t.html
- 规则 #4：表达式中无符号数的等级更低时，如果有符号数的类型无法完全覆盖无符号数，则将它们转换为有符号数的类型对应的无符号数
  - 例如：在常见的 ILP32 数据模型下，`sizeof(int) == sizeof(long) == 4`
    - 比较 `1u` 和 `-1l` 时
    - 因为 `unsigned` 的等级低于 `long`，但 `long` 不能覆盖 `unsigned` 的范围（重要）
    - 所以都会把他们转化为 `unsigned long` 比较。

整型/类型提升

```c
#include <stdio.h>

int main() {
  unsigned char uc = 128;
  signed char c = -128;
  printf("%d %d\n", uc == c, uc + c);
}
```

1. `uc` 的初始值为 128，二进制表示为 `1000 0000`；`c` 的初始值为 -128，在补码机器上的二进制表示也为 `1000 0000`。
2. 二者进行运算时，都被扩展到 `int`。`uc` 做无符号数的零扩展然后改变解释方式，变为 `... 1000 0000`，值仍为 128。
3. `c` 做有符号数扩展，前面补 1，变为 `1111 ... 1000 0000`，值为 -128。
4. 所以输出为 `0 0`。

整型/类型提升

Integral/Type Promotion

请分别回答以下程序在 ILP32 和 LP64 数据模型下的输出是什么

```c
#include <stdio.h>

int main() {
  unsigned short us = 1;
  printf(
    "%d %d\n",
    1u + 1l > -1l,
    2147483646 + us > -2147483647 - 1
  );
}
```

ILP32

应该还是比较简单的

- `1u` 和 `1l` 相加得到 `2ul`，小于 `-1ul`。
- `int` 能完整表示 `short`，所以 `1us` 直接被扩展到 `1`
- 所以 lhs 为 2147483647，正好是 `INT_MAX`，rhs 正好是 -2147483648，正好是 `INT_MIN`。

LP64

- 此时，`long` 是 64 位的，表示范围能覆盖 `unsigned`，所以第一个表达式统一扩展到 `long`，`1u + 1l` 结果为 `2l`。
- 后者不变

练习：类型转换

Exercises

在 x86-64 机器上，对任意的整型 x 和 y 值，ux 和 uy 分别为其转化成无符号数的值，则下面的 C 语言表达式中，等价的是（不成立的给出反例）

命题 1	命题 2	等价?	原因/反例
`x > y`	`ux > uy`	False	`x = 0, y = -1`
`(x > 0) || (x < ux)`	`1`	False	`x <= 0`
`x ^ y ^ x ^ y ^ x`	`x`	True	异或的性质
`((x >> 1) << 1) <= x`	`1`	True	右移后再左移，清空最低位
`((x / 2) * 2) <= x`	`1`	False	向零取整，负奇数
`x ^ y ^ (~x) - y`	`y ^ x ^ (~y) - x`	True	运算符优先级
`(x == 1) && (ux - 2 < 2)`	`(x == 1) && ((!!ux) - 2 < 2)`	False	左侧 `ux` 是 `unsigned int` 类型，但右侧 `!!ux` 会被自动转换为 int

练习：浮点运算

Exercises

考虑 IEEE-754 标准的浮点数

Description	True?	Why?
对于任意单精度浮点数 `a` 和 `b`，若 `a > b`，则 `a + 1 > b`	True	显然
对于任意单精度浮点数 `a` 和 `b` 和 `c`，若 `a > b`，则 `a + c > b + c`	False	`inf`
对于任意双精度浮点数 `d`，若 `d < 0`，那么 `d * d > 0`	False	下溢
对于任意双精度浮点数 `d`，若 `d < 0`，那么 `d * 2 < 0`	True	`-inf` 也没关系
对于任意双精度浮点数 `d`，`d == d`	False	NaN
将 `float` 转换为 `int` 时，既可能造成舍入，也可能造成溢出	True	显然

练习：浮点运算

Exercises

（19 期中）运行下面的代码，输出结果是（其中 `float` 类型表示 IEEE-754 规定的浮点数，包括 1 位符号、8 位阶码和 23 位尾数）：

```
for (float f = 1; ; f = f + 1) {
  if (f + 1 - f != (float)1) {
    printf("%.0f\n", f);
    break;
  }
}
```

A. `8388608(=2^23)` B. `16777216(=2^24)` C. `2147483647(=2^31-1)` D. 程序为死循环，没有输出

`float` 有几位尾数？答案很明显了。

练习：浮点舍入

Exercises

（13 期中）对 $x = 1\frac{1}{8}$ 和 $y = 1\frac{3}{8}$ 进行小数点后两位取整（rounding to nearest even），结果正确的是

A. $1\frac{1}{4}, 1\frac{1}{4}$

B. $1, 1\frac{1}{4}$

C. $1\frac{1}{4}, 1\frac{1}{2}$

D. $1, 1\frac{1}{2}$

$x = (1.00100)_2$，“四舍六入五成双”，$(1.00)_2$。

$y = (1.01100)_2$，“四舍六入五成双”，$(1.10)_2$。

小测试

quiz

下面关于布尔代数的叙述，错误的是：

- A. 设 `x，y，z` 是整型，则 `x^y^z == y^z^x`
- B. 若允许使用“全 1”常量，任意逐位布尔运算都可以由与运算（`&`）和异或运算（`^`）组合得到
- C. 设 `m，n` 是 `char*` 类型的指针，则下面三条语句 `*n = *m^*n; *m = *m^*n; *n = *m^*n;` 可以交换 `*m` 和 `*n` 的值
- D. 已知 `a，b` 是无符号整型，且 `a+b+1==0` 为真，则 `a^b+1==0` 为真

A. 正确，考虑每个 bit 的最终结果只和 `x，y，z` 的对应 bit 有多少个 0、多少个 1 有关，而和异或顺序无关

B. 正确。记 `ALL_ONES` 为与操作数等宽的全 1 常量，则 `~A = A ^ ALL_ONES`，并可由德摩根律构造 `A | B = ~(~A & ~B)`。

C. 错误，因为 `m` 和 `n` 可能指向同一个地址，所以第一句话直接置零了。但对于其他情况，是对的。

D. 正确，这个非常离谱，下一页 PPT 单独讲。

小测试

quiz

- D. 已知 `a, b` 是无符号整型，且 `a+b+1==0` 为真，则 `a^b+1==0` 为真

第一层理解：如果先按直觉给右式加上括号，前提在无符号模运算下给出 `a + b == UINT_MAX`，因此 `a == ~b`，从而 `(a ^ b) + 1 == 0`。

第二层理解：右式没有括号。根据 优先级表，计算优先级为 `+` > `==` > `^`，所以实际表达式是 `a ^ ((b + 1) == 0)`，需要重新判断。

第三层理解：在第二层理解上再次思考，考虑了优先级后，何时 `a^((b+1)==0)` 为假？

该异或表达式只有在 `a` 等于布尔值 `0` 或 `1` 时才可能为 0；同时前提给出 `a + b == UINT_MAX`。

`a = 1`，则 `b = UINT_MAX - 1`，所以 `(b + 1) == 0` 为假，最终 `a ^ 0 == 1`。

`a = 0`，则 `b = UINT_MAX`，所以 `(b + 1) == 0` 为真，最终 `a ^ 1 == 1`。

所以，此选项仍然正确。



Thank you for your listening!

2026/10/1

© SHANGHAI HENIAN INFORMATION TECHNOLOGY CO.,LTD